

DAY 2 · MODULE 4 · 35 MIN

Tools Layer Deep Dive

How agents do things beyond talking

Where we are in the stack

Modules 1–3 covered Knowledge. Modules 4–7 cover Actions.

- Model — your Foundry-deployed model
- Runtime — Prompt agent, Hosted agent, your own code + Responses API
- Actions ← Modules 4–7
- Knowledge — Modules 1–3
- Ops — Day 5

Actions = how the agent DOES things. Retrieval = how it KNOWS things.

Module 4's job

This module is the framing module for Actions.

- Module 4 (this one) — what function calling is, tool schema, error contracts, streaming — the concepts
- Module 5 — Foundry Toolbox (managed tools you attach)
- Module 6 — authoring your own function tools in MAF

You leave Module 4 knowing HOW tool calling works under the hood. Module 6 turns that into code.

The function-calling model in one picture

User: "What's the weather in Amsterdam?"

- Agent calls model with:
 - system prompt (instructions)
 - user message
 - list of tool schemas [{name, description, params}, ...]
- Model returns: `tool_call(name="get_weather", args={"location": "Amsterdam"})`
- Agent invokes `get_weather("Amsterdam")` LOCALLY, gets: "cloudy, 15°C"
- Agent calls model again with tool result appended
- Model returns: "The weather in Amsterdam is cloudy with a high of 15°C."

The model never runs your code. It emits INTENT — your process runs the code. This is the fundamental contract.

Why this works — the model as decision-maker

The model doesn't know how to check the weather. It knows:

- When to use a weather tool (based on the tool description)
- What arguments to pass (based on the parameter schema and descriptions)
- What to do with the result (based on the tool's return value and shape)

The model is picking from a menu of tools you defined. Your tool schema IS the menu. Bad schema = bad picks.

Tool schema anatomy

Every function tool exposes four things to the model:

Field	What the model uses it for
Name	Referenced when calling — should be a clean identifier
Description	The primary 'when to call this' signal
Parameter schema	Types, required/optional, per-param descriptions
Return type	What comes back after the tool runs

The description carries more weight than you might realize. Modules 3 and 6 both come back to this.

In MAF — the shortest possible tool

Python — any function is a tool:

```
def get_weather(
    location: Annotated[str, Field(description="The location to get the weather for.")],
) -> str:
    """Get the weather for a given location."""
    return f"The weather in {location} is cloudy with a high of 15°C."

agent = Agent(
    client=FoundryChatClient(credential=AzureCliCredential()),
    instructions="You are a helpful weather assistant.",
    tools=[get_weather],
)
```

Docstring → description. Field(description=...) → parameter description. Type hints → schema.

Module 6 goes deeper on authoring. Note the shape now.

In MAF — the C# equivalent

Same shape, different syntax:

```
[Description("Get the weather for a given location.")]
static string GetWeather(
    [Description("The location to get the weather for.")] string location)
    => $"The weather in {location} is cloudy with a high of 15°C.";

AIAgent agent = new AIProjectClient(new Uri(endpoint), new DefaultAzureCredential())
    .AsAIAgent(
        model: "gpt-5.6-luna",
        instructions: "You are a helpful assistant",
        tools: [AIFunctionFactory.Create(GetWeather)]);
```

Attribute-based description; AIFunctionFactory.Create wraps the method. Same primitives, same model-facing contract.

The @tool decorator — when you want control

Explicit name, description, and other options via @tool:

```
from agent_framework import tool

@tool(name="weather_tool", description="Retrieves weather information for any location")
def get_weather(
    location: Annotated[str, Field(description="The location to get the weather for.")],
) -> str:
    return f"The weather in {location} is cloudy with a high of 15°C."
```

- The function's Python name isn't what you want the model to see
- You want the description in one place (not a docstring)
- You need parameters like `approval_mode` (Day 5) or explicit schemas (next slide)

Explicit schemas

When you need full control, pass a Pydantic model or a raw JSON schema:

```
class WeatherInput(BaseModel):  
    location: Annotated[str, Field(description="The city name")]  
    unit: Annotated[str, Field(description="celsius or fahrenheit")] = "celsius"  
  
@tool(name="get_weather", description="Get current weather.", schema=WeatherInput)  
def get_weather(location: str, unit: str = "celsius") -> str:  
    return f"Weather in {location} is 22 degrees {unit}."
```

- Schema documented in one place (not spread across type hints)
- Generating tools programmatically
- Need enum constraints, min/max, or validation type hints can't express

Runtime context — hidden from the model

Some values shouldn't be model-visible: the calling user, session, DB handle.

```
@tool(approval_mode="never_require")
def get_weather(
    location: Annotated[str, Field(description="The location to get the weather for.")],
    ctx: FunctionInvocationContext,
) -> str:
    user_id = ctx.kwargs.get("user_id", "unknown")
    return f"The weather in {location} is cloudy with a high of 15°C."

await agent.run("What's the weather in Amsterdam?",
    function_invocation_kwargs={"user_id": "user_123"})
```

ctx is injected by the framework. Hidden from the schema the model sees. Use for logging, personalization, tenancy.

Tool description patterns that work

The description is a mini-prompt. Same discipline as Day 1 Module 3.

- What it does — one clear sentence
- When to use it — the condition that triggers this tool
- When NOT to use it — differentiate from other tools
- Parameter descriptions — meaning, valid values, examples

```
@tool
def create_ticket(title: str, body: str, priority: str) -> str:
    """Create a support ticket for a problem that needs a human engineer.

    Use this when the user reports a problem you cannot answer from
    documentation. Do NOT use for questions you can answer directly.

    Priority must be one of: low, med, high.
    """
```

Streaming tool progress

For long-running tools, don't block the user. MAF streams tool-call events alongside model tokens:

```
async for event in agent.run("Look up my account status", stream=True):  
    if event.type == "tool_call_start":  
        print(f"→ calling {event.tool_name}({event.args})")  
    elif event.type == "tool_call_result":  
        print(f"← {event.tool_name} returned")  
    elif event.text:  
        print(event.text, end="", flush=True)
```

Tool call, tool result, tokens — all in one async stream. Day 3 covers streaming UX and cancellation.

Error contracts

Your tool will fail. What does it return?

- String describing the error — model can incorporate ('couldn't reach the service, try again')
- Structured error object — model can retry with different args or route to a different tool
- Raise an exception — MAF surfaces it back to the model with the exception message

Rule of thumb: return errors as data when you want the model to recover. Raise when the failure is unrecoverable.

Day 3 covers robust agents in depth — retries, timeouts, guardrails.

Approval mode

For tools with side effects (create ticket, send email, delete record), you don't always want the model to fire them autonomously.

```
@tool(approval_mode="always_require")
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email. Requires user approval before firing."""
    ...
```

- never_require — safe / idempotent tools (read-only, no side effects)
- always_require — every call needs human approval
- Heuristic — MAF's default: approve safe patterns, prompt on risky ones

Day 5 (Responsible AI) revisits this for production HITL patterns.

Three kinds of tools you'll encounter

Kind	What it is	Where
Function tool	Your Python or C# code, exposed to the model	Module 6 today
Foundry Toolbox tool	Managed tool in the Foundry catalog (Bing, code interpreter, SharePoint...)	Module 5 today
MCP tool	Any tool exposed by an MCP server, local or remote	Day 3

All three go through the same function-calling contract. Same schema shape. Same tool call → tool result flow.

Differences: who wrote the code, where it runs, who authenticates it.

The tool-vs-knowledge decision

You'll build things where either could work: 'should I make this a tool call to search docs, or attach a knowledge base?'

- Tool when the retrieval is one option among many (agent decides)
- Knowledge when the retrieval should happen on every relevant query (retrieval is grounding, not action)

You can have both. Common pattern: knowledge base for background grounding + function tools for actions. Use clear instructions and tool descriptions to steer which fires when.

Common traps

- Vague tool descriptions — 'gets data' — model has no signal for when to call
- Overlapping tools — two tools with fuzzy descriptions; model picks wrong one
- Too many tools — 30 tools registered; model gets confused. Rule: <10 per agent.
- Side effects in 'read' tools — `check_status` also logs, mutates, or bills. Keep side effects in write-tagged tools.
- Missing parameter descriptions — model guesses arg values. Bad guesses = bugs.
- Silent exceptions — tool raises, model doesn't know why. Return errors as data.

Takeaways

- Function calling = the model picks tools from a menu you defined. Bad menu = bad picks.
- Tool schema is a prompt. Name, description, parameter descriptions all matter.
- In MAF, any Python function or C# method can be a tool — decorators and attributes give finer control.
- Runtime context via ctx keeps sensitive values out of the model.
- Return errors as data, not exceptions, when you want the model to recover.
- All tools (function, Toolbox, MCP) share the same contract — different origins, same shape.

Next: Foundry Toolbox — managed tools you attach without writing.